

Exploring the Social World - Quantitative Block: Statistics

Gabriele Filomena and Zi Ye

2026-09-28

Table of contents

Welcome	5
Contact	5
Overview	6
Aim and Learning Objectives	6
Module Structure	7
Software and Data	8
Setup: RStudio in your Browser	9
1. First time only	9
2. Every time you work	10
3. Your files	10
4. Your first lab (Week 1)	11
5. Adding data from Canvas	11
6. Saving a copy on your computer	12
7. If something goes wrong	12
Another option: RStudio on a university PC or your own laptop	13
Setup: RStudio on a PC or Laptop	14
1. Get RStudio	14
2. Get the course files	14
3. Open the course in RStudio	15
4. Install the packages (once)	15
5. Work on the labs	15
6. Adding data from Canvas	16
7. Getting updated labs	16
8. If something goes wrong	16
Assessment	17
Required Report Structure	17
How to get there?	18
How is it graded?	19
Assessment: How to submit	28
.html file	28

Assessment Template	29
Setup	29
Introduction	31
Literature Review	31
Methods and Data	31
Data	31
Variables	32
Transformations	32
Analysis	32
Results and Discussion	32
Descriptive statistics	33
Regression model	33
Discussion	34
Conclusion	34
References	34
 Samples, Uncertainty and P-values	 35
Setting Up	35
Why Sample Size Matters	35
Confidence Intervals	36
Seeing What “95%” Means	37
Significance and Sample Size	40
What a P-value Is Not	41
A Note on Census Data	41
 1 Lab: Introduction to R	 42
1.1 R and RStudio	42
1.1.1 The four panels	42
1.2 Quarto documents and code chunks	43
1.3 R as a calculator	45
1.3.1 Functions and arguments	46
1.3.2 Getting help	46
1.4 Variables	47
1.5 Packages	47
1.6 Next	47
 2 Lab: Exploring a Dataset	 48
2.1 Practice: Dataset and Dataframes	48
2.1.1 Datasets in R	49
2.1.2 Importing Data in R	50
2.2 Practice: Descriptive Statistics	53
2.2.1 Summarizing Data	53
2.2.2 Understanding the Structure of the FRS Datafile	60

2.2.3	Explore the Distribution of Your Outcome Variable	61
2.3	Practice: From Sample to Population	69
2.3.1	Sampling Variability	70
2.3.2	The Standard Error	71
2.3.3	What a P-value Means	72

Welcome

This is the website for “Exploring the Social World - Quantitative Block: Statistics” (module **ENVS225**) at the University of Liverpool. This block of the module is designed and delivered by Dr. Gabriele Filomena and Dr. Zi Ye from the Geographic Data Science Lab at the University of Liverpool. The module seeks to provide hands-on experience and training in introductory statistics for human geographers.

The website is **free to use** and is licensed under the [Attribution-NonCommercial-NoDerivatives 4.0 International](#). A compilation of this web course is hosted as a GitHub repository that you can access:

- As an [html website](#).
- As a [GitHub repository](#).

Contact

Gabriele Filomena - gfilo [at] liverpool.ac.uk Lecturer in Geographic Data Science
Office 1xx, Roxby Building, University of Liverpool - 74 Bedford St S, Liverpool,
L69 7ZT, United Kingdom.

Zi Ye - zi.ye [at] liverpool.ac.uk Lecturer in Geographic Information Science Office
107, Roxby Building, University of Liverpool - 74 Bedford St S, Liverpool, L69
7ZT, United Kingdom.

Overview

Aim and Learning Objectives

This sub-module aims to provide training and skills on a set of basic quantitative research methods for data collection, analysis, and interpretation. You will learn how to define coherent, relevant research questions, utilise various research quantitative methods, and identify appropriate methodologies to tackle your research questions. **This block serves as the foundation for the dissertation and fieldwork modules.**

Background

Data and research are key pillars of the global economy and society today. We need rigorous approaches to collecting and analysing both the statistics that can tell us ‘how much’ and if there are observable relationships between phenomena; and the information gives us a nuanced understanding of cultural contexts and human dynamics. Quantitative skills enable us to explore and measure socio-economic activities and processes at large scales, while qualitative skills enable understanding of social, cultural, and political contexts and diverse lived experiences. Rather than being in opposition, qualitative and quantitative research can complement one another in the investigation of today’s pressing research questions.

To these ends, this block will help you develop your quantitative (statistical) skills, as critical tools. This course will help you understand what quantitative statistical researchers use and develop a set of research techniques that can be used in your field classes and dissertations.

Learning objectives:

- Understand how to explore a dataset, containing a number of observations described by a set of variables.
- Demonstrate an understanding in the application and interpretation of commonly used quantitative research methods.
- Demonstrate an understanding of how to work with quantitative data to address real-world research questions.

Module Structure

Staff: Dr Zi Ye and Dr Gabriele Filomena

Where and When

Week 1-6 Monday @ Central Teaching Hub PCTC

- Lecture (9 – 10 am).
- Practical PC session (10 am – 12 pm).

Lectures will introduce and explain the fundamentals of quantitative methods, with the opportunity to apply the method introduced in the practical session straight after.

The computer practical sessions, will give you the chance to use and apply quantitative methods to real-world data. These are primarily self-directed sessions, but with support on hand if you get stuck. Support and training in R will be provided through these sessions. Weekly sessions will be driven by empirical research questions.

Week	Topic	Format	Staff
1	Introduction & Review	Lecture and Computer Lab Practical	GF
2	Simple & Multiple Linear Regression	Lecture and Computer Lab Practical	GF
3	Multiple Linear Regression with Categorical Variables	Lecture and Computer Lab Practical	ZY
4	Logistic Regression	Lecture and Computer Lab Practical	ZY
5	Data Visualisation	Lecture and Computer Lab Practical	GF
6	Summary and Assessment Support	Lecture and Computer Lab Practical	ZY

Software and Data

We work in **RStudio in your web browser** (GitHub Codespaces): R, RStudio, the packages and the course data are ready to use, with nothing to install. **Follow the [Setup](#) guide before the first practical.**

RStudio is also installed on University Machines, and you can install [R](#) and [RStudio Desktop](#) on your own laptop if you prefer (see the end of the [Setup](#) guide).

This software is already installed on University Machines. But you will need it to run the analysis on your personal devices.

Data

Example datasets could be accessed through Canvas or (some) on [GitHub](#) Repository of the module. These include:

- 2021 UK Census Data.
- 2021 Annual Population Survey (APS) - only on Canvas.
- 2016 Family Resource Survey (FRS) - only on Canvas.
- 2011 Sample of Anonymised Records (SAR).

Note: The Annual Population Survey requires the completion of a quiz prior to its usage, as it is licensed.

Setup: RStudio in your Browser

In this module you use **RStudio in your web browser**. You don't need to install anything. R, RStudio and all the course files are ready for you.

This works on any computer: university PCs, Windows laptops, Macs and Chromebooks.

It runs on **GitHub Codespaces**. A *codespace* is your own computer on the internet. Only you can use it, and it keeps your files between sessions.

1. First time only

Step 1. Make a GitHub account.

Go to github.com/signup and create a free account. You can use your university email. Then sign in.

Step 2. Create your codespace.

Open this link, then click the green **Create codespace** button:

<https://codespaces.new/GDSL-UL/stats?quickstart=1>

Wait while your codespace is built. **The first time this takes several minutes**, because R, RStudio and the course packages are installed for you. Don't close the tab. (Next time it opens much faster.)

When it's ready, a blue and grey window opens in your browser. This is called VS Code. **You don't need to use it**, but keep it open in the background.

Step 3. Open RStudio.

When everything is ready, you have two ways to open RStudio:

- A small message appears in the bottom-right corner. Click **Open in Browser**.
- Or look at the panel at the bottom of the window. Find the box that says **RStudio is ready**. Hold Ctrl (Cmd on a Mac) and click the link in the box.

RStudio opens in a new tab.



Tip

The new tab is stuck on a GitHub page that says “authenticating”? This is normal. Try these in order:

1. Press F5 to refresh the page.
2. Wait a minute: RStudio may still be starting. Then refresh again.
3. Use a normal browser window, not a private or incognito one. Chrome or Edge work best. Some browsers (Safari, Brave, or Firefox/Edge with strict tracking protection) block the cookie GitHub needs to sign you in: allow cookies for `github.dev` or switch browser.
4. In the VS Code window, click the **Ports** tab in the bottom panel, right-click the **RStudio** row (port 8787) and choose **Open in Browser**.

Step 4. Bookmark the RStudio tab.

This is your personal RStudio address. Use it to come back quickly next time.

2. Every time you work

1. Go to github.com/codespaces and click your codespace. You can also click the button in Step 2 again: it takes you back to the codespace you already have.
2. Open RStudio with your bookmark, or as in Step 3.
3. Work in the RStudio tab.
4. When you finish, go back to github.com/codespaces. Click the ... next to your codespace and choose **Stop codespace**. This saves your free hours. Your files are not deleted.

3. Your files

In RStudio, the **Files** panel (bottom right) shows the course folder:

```
stats/  
  data/    the datasets  
  img/     pictures used in the labs  
  labs/    one lab file for each week  
  myLabs/  your own copies of the labs: you work here
```

Everything is already in the right place. **You don’t need to download, move or set up anything.**

The **labs** folder updates automatically each time you open your codespace, so you always have the latest version of the labs. **Don't work in the files in labs:** if you change one, the whole **labs** folder can stop updating. Work on your own copy in **myLabs** instead (see below). Nothing in **myLabs** is ever changed or replaced.

4. Your first lab (Week 1)

1. In the **Files** panel, click **labs**, then click **00.introR.qmd**. The lab opens in the top-left panel. It shows you around RStudio and how to run code.
2. **Make your own copy.** Click **File -> Save As...**, go to the **myLabs** folder, and click **Save**. The copy is now open in RStudio: work in this one.
3. Then open **01.DataExploration.qmd** from **labs** and copy it to **myLabs** in the same way. This is the main practical for Week 1.
4. Go through each lab from top to bottom and run each code chunk. Write your notes and answers in the white space between the chunks.
5. Save often: press **Ctrl+S** (**Cmd+S** on a Mac).

Each week, open that week's lab from **labs**, save a copy in **myLabs**, and work on the copy. Next time, open your copy straight from **myLabs**.

Your changes are only saved in your own codespace. They don't change the website, and other students can't see them.

Note

How the labs find the data

The lab code loads data like this:

```
attacks <- read.csv("../data/attacksUK.csv")
```

`../` means "go up one folder". Your copy of the lab is in **myLabs**, so R goes up to **stats** and then into **data**. You don't need to change these lines.

5. Adding data from Canvas

Some datasets are only on Canvas, for example the Family Resource Survey (FRS). To add one:

1. Download the file from Canvas to your computer.
2. In RStudio's **Files** panel, click **data**.

3. If the lab asks for a new folder (for example **FRS**), click **New Folder**, type the name, and click **OK**. Then click the new folder to open it.
4. Click **Upload**, then **Choose File**. Pick the file you downloaded and click **OK**.

Now the lab code will find the file.

6. Saving a copy on your computer

To download a file from RStudio (for example to keep a copy, or to submit your work):

1. In the **Files** panel, tick the box next to the file.
2. Click **More**, then **Export...**, then **Download**.

For the assessment, download the report template from Canvas and upload it to your **myLabs** folder (the same way as in section 5). Write your report there. You submit an **.html** file, which you make with the **Render** button at the top of your file. See [How to submit](#).

Warning

Keep copies of your work. If you don't open your codespace for 30 days, GitHub deletes it, and your files are deleted too. Download your files from **myLabs** from time to time, and always before holidays and before the assessment deadline.

7. If something goes wrong

- **RStudio is stuck on “authenticating”.** Press F5 to refresh the page. If that doesn't help, see the tips in Step 3 of section 1.
- **RStudio says it lost the connection.** Your codespace has probably stopped. Open it again from github.com/codespaces, then refresh the RStudio tab.
- **You don't see the myLabs folder.** Your codespace was created before we added it. In the VS Code window, click in the panel at the bottom, type `git pull` and press Enter. Then stop your codespace and open it again (see section 2).
- **R says it can't find a file.** Check that the file is really in the folder named in the code. You can see this in the **Files** panel.
- **GitHub says you have no free hours left.** The free plan gives you about 60 hours a month. Always stop your codespace when you finish (see section 2). If you run out, use RStudio on a university PC (see [Setup: RStudio on a PC or Laptop](#)).

Another option: RStudio on a university PC or your own laptop

You can also work in RStudio installed on a university PC or on your own laptop. See [Setup: RStudio on a PC or Laptop](#).

Setup: RStudio on a PC or Laptop

The easiest way to work in this module is [RStudio in your browser](#). You can also use RStudio installed on a computer:

- on a **university PC**, where RStudio is already installed;
- on **your own laptop**, after installing R and RStudio.

This is useful if you run out of free codespace hours, if you want to work offline, or if a data file is too big to upload to your codespace.

The labs are the same. The differences are that you set things up once yourself, and the labs don't update automatically.

1. Get RStudio

University PC. R and RStudio are already installed. Go to step 2.

Your own laptop. Install these two programs, in this order:

1. [R](#): pick the version for your system (Windows, macOS or Linux) and install it with the default options.
2. [RStudio Desktop](#): download the free version and install it.

2. Get the course files

1. Go to the [course page on GitHub](#). Click the green **Code** button, then **Download ZIP**.
2. **Unzip the file** (on Windows: right-click it, then **Extract All...**). Don't work inside the ZIP file: R can't find the data there.
3. Put the unzipped **stats** folder somewhere you can find it. On a university PC, put it in your M: drive, so you can open it from any PC on campus.

The folder also contains the website and the lecture slides. You only need these parts:

```
stats/  
  data/           the datasets  
  img/            pictures used in the labs  
  labs/           one lab file for each week  
  myLabs/         your own copies of the labs: you work here  
  envs225.Rproj   opens the course in RStudio
```

3. Open the course in RStudio

Double-click `envs225.Rproj` in the `stats` folder. RStudio opens, and the **Files** panel (bottom right) shows the course folder.

Next time, open the course the same way, or in RStudio click `File -> Recent Projects` and choose `envs225`.

4. Install the packages (once)

In the codespace all the packages are already installed. On a PC or laptop you install them once. Copy this line into the **Console** (bottom left), press Enter, and wait until it finishes. This can take a few minutes.

```
install.packages(c("tidyverse", "broom", "knitr", "rmarkdown", "scales", "readxl", "RColorBr"))
```

If RStudio asks whether to install from sources, or to use a personal library, answer **No** to sources and **Yes** to the personal library.

On a university PC you may need to do this again if you log in to a different PC. If a lab stops with `there is no package called ...`, install that package with `install.packages("name")`, or use `Tools -> Install Packages...`

5. Work on the labs

Work as in the codespace (see [Setup](#), section 4):

1. In the **Files** panel, click `labs` and open that week's lab.
2. Click `File -> Save As...`, go to the `myLabs` folder, and click **Save**. Work on this copy.
3. Go through the lab from top to bottom and run each code chunk.
4. Save often: press `Ctrl+S` (`Cmd+S` on a Mac).

The lab code loads data with paths like `"../data/attacksUK.csv"`. These work because your copy is in `myLabs`, next to `data`. You don't need to change them.

6. Adding data from Canvas

Download the file from Canvas and move it into the **data** folder with File Explorer (Windows) or Finder (Mac). If the lab asks for a new folder (for example **FRS**), create it inside **data** first.

There is no upload step and no size limit, so this also works for large files.

7. Getting updated labs

In the codespace the **labs** folder updates by itself. On a PC or laptop it doesn't. If we tell you a lab has changed:

1. Download the ZIP again and unzip it (step 2).
2. Copy its **labs** folder into your **stats** folder and replace the old one.

Never replace your myLabs folder: your work is there.

8. If something goes wrong

- **R says it can't find a file.** Check that you unzipped the course files (step 2), that you opened the lab from **myLabs**, and that the data file is really in the folder named in the code.
- **there is no package called ...** Install the package (step 4).
- **The Render button asks to install packages or tools.** Click **Yes** and let RStudio install them.

Assessment

Deadline: Thursday 12th November 2026 - 2pm. **Word count:** 2000 words - including tables, excluding references. **Format:** a single `.html` file rendered from your `.qmd` (see [How to submit](#)).

The assignment **Data Exploration and Analysis** consists of writing a research report using one of the regression techniques learned during the module. The basic idea is to put in practice the methods learned during the quantitative block of the module. You are required to apply a linear or logistic regression model to the data provided for the module. The report needs to include the following sections (in brackets, % of the whole length):

- Introduction (5%).
- Literature Review (20%).
- Methods and data (30%).
- Results and discussion (40%).
- Conclusion (5%).
- Reference List.

Required Report Structure

1. Introduction

- Context: Why is the topic relevant or worth being investigated?
- Brief discussion of existing literature.
- Knowledge gap and Aim.
- 1 Research question.

2. Literature review

- More detailed Literature review, i.e. what do we already know about this subject
- Rationale for including certain predictor variables in the model.
- What knowledge gap remains that this article will address? (includes “not studied before in this area”). *Note: there is no expectation on totally original research. The focus is on a clean, sensible, data analysis situated in existing ideas.*

3. Methodology:

- A brief introduction to the dataset being analysed (who collected it? When? How many responses? etc.)
- A description of the variables chosen to be analysed.
- A description of any transformation made to the original data, i.e. turning a continuous variable of income into intervals, or reducing the number of age groups from 11 to 3.
- A description and justification of the statistical techniques used in the subsequent analysis (i.e. the multivariate regression model: Multiple Linear or Logistic Regression).

4. Results and Discussion

- Descriptive statistics and summary of the variables employed, supported by tables or/and charts.
- Correct interpretation of correlation coefficients, statistical significance, and model fit.
- Usage and results of an appropriate multivariate regression model.
- Interpretation of the results, including links and contrasts to existing literature.
- Selective illustrations (graphs and tables) to make your findings as clear as possible.

5. Conclusion

- Summary of main findings.
- Limitations of study (self-critique).
- Highlight any implications derived from the study.

Follow this structure and include **ALL** these points, do not make your life harder.

See [example assignments](#) from previous years on Canvas (you need to be logged in with your UoL account).

How to get there?

The first stage is to identify **ONE** relevant research question to be addressed. Based on the chosen question, you will need to identify a dependent (or outcome) variable which you want to explain, and at least two relevant independent variables that you can use to explain the chosen dependent variable. The selection of variables should be informed by the literature and empirical evidence.

To detail in the Methods Section: Once the variables have been chosen, you will need to describe the data and **appropriate** type of regression to be used for the analysis. You need to explain any transformation done to the original data source, such as reclassifying variables, or changing variables from continuous to nominal scales. You also need to briefly describe the data use: source of data, year of data collection, indicate the number of records used, state if

you are using individual records or geographical units, explain if you are selecting a sample, and any relevant details. You also need to identify type of regression to be used and why.

To detail in the Results and Discussion Section: Firstly, you need to provide two types of analyses. First, you need to provide a descriptive analysis of the data. Here you could use tables and/or plots reporting relevant descriptive statistics, such as the mean, median and standard deviation; variable distributions using histograms; and relationships between variables using correlation matrices or scatter plots. Secondly, you need to present an estimated regression model or models and the interpretation of the estimated coefficients. You need a careful and critical analysis of the regression estimates. You should think that you intend to use your regression models to advice your boss who is expecting to make some decisions based on the information you will provide. As part of this process, you need to discuss the model assessment results for the overall model and regression coefficients. Remember to substantiate your arguments using relevant literature and evidence, and present results clearly in tables and graphs.

How is it graded?

Grade	Score Range	UG	Descriptor	Assignment Expectations
Fail	0-34%	Fail	Inadequate	Literature Review: Lacks relevance and fails to justify variable choice. Evidence is irrelevant or missing, providing no support to the research question. Methods: Data is not described, and the regression model is entirely missing. No appropriate statistical method is applied. Results and Discussion: No descriptive statistics, graphs, or tables are provided. Model results and interpretation are absent. Structure and References: Report is disorganized with significant referencing and citation errors throughout.

Grade	Score Range	UG	Descriptor	Assignment Expectations
Narrow Fail	35-39%	Fail	Highly Deficient	Literature Review: Review is present but lacks coherence and fails to justify variable choice. Evidence is poorly aligned with the research question and mostly irrelevant. Methods: Minimal data description; the regression model is missing but some statistical methods are mentioned. Results and Discussion: Few or no descriptive statistics or visuals are present. Statistical methods are unclear or incorrectly applied. Results are vague and lack meaningful interpretation. Structure and References: Report structure is poor, with referencing errors in multiple sections.

Grade	Score Range	UG	Descriptor	Assignment Expectations
Third / Fail	40-49%	Third (UG)	Deficient	Literature Review: Relevant literature is partially addressed but lacks depth, with limited justification for variable choice. Evidence is minimally aligned with the research question. Methods: A very basic data description is provided, but the selected regression model is deeply inadequate or incorrect (e.g., multiple linear regression for a categorical outcome; logistic regression for a continuous outcome). Results and Discussion: Descriptive statistics or visuals may be present but insufficient. Model results are presented with little to no interpretation. Structure and References: Report structure is present but lacks clarity, with inconsistencies

Grade	Score Range	UG	Descriptor	Assignment Expectations
2.2 / Pass	50-59%	2.2 (UG)	Adequate	Literature Review: Addresses relevant literature but with limited justification of variable choices. Evidence generally supports the research question but lacks detail. Methods: Data description is present but brief; a regression model is included but applied illogically or incorrectly (e.g., multiple linear regression for a categorical outcome; logistic regression for a continuous outcome) and with little explanation. Results and Discussion: Basic descriptive statistics, graphs, or tables are presented; the regression model is applied with some inaccuracies and/or interpretation is minimal. Structure and References: Report is mostly organized

Grade	Score Range	UG	Descriptor	Assignment Expectations
2.1 / Merit	60-69%	2.1 (UG)	Good	Literature Review: Relevant literature is discussed, with some justification for variable choice. Evidence supports the research question well. Methods: Data is described with some detail, though potential data transformations are under-explored. The regression model is appropriate for the selected variable types. Results and Discussion: Descriptive statistics and visuals are provided. Model results are discussed, though interpretation lacks depth. Findings are compared to existing literature. Structure and References: Report is logically structured and clear, with mostly correct citations.

Grade	Score Range	UG	Descriptor	Assignment Expectations
First / Distinction	70-79%	First (UG)	Very Good	Literature Review: Strong grasp of relevant literature, with well-justified variable selection. Evidence aligns well with the research question. Methods: Data is comprehensively described with consideration of relevant transformations. The regression model is appropriate and well-justified. Results and Discussion: Descriptive statistics and clear visuals support findings. Model results are accurately interpreted with strong connections to existing literature. Structure and References: Report has a coherent, professional structure with only minor referencing errors.

Grade	Score Range	UG	Descriptor	Assignment Expectations
High First / High Distinction	80-100%	High First (UG)	Excellent to Outstanding	Literature Review: Critical and thorough literature review with strong, well-justified variable selection. Evidence fully supports the research question with insightful connections. Methods: Detailed data description and transformation steps are clearly articulated. Regression model is expertly applied and justified. Results and Discussion: Comprehensive descriptive statistics, graphs, and tables are provided. Model results are innovatively interpreted with strong links to existing research. Structure and References: Report is professionally structured, with flawless citations and a high standard of organization.

In summary:

1. **Introduction:** Should establish the topic's relevance, present a concise literature overview, identify a knowledge gap, and outline research questions.
2. **Literature Review:** Requires an in-depth review of relevant studies, justification for chosen independent variables, and identification of a potential knowledge gap or unexplored area aligned with the chosen research question.
3. **Methods and Data:** Should describe the dataset, variable transformations, and justify the regression technique. Key transformations, such as reclassifying variables, should be explained with clarity and relevance.
4. **Results and Discussion:** Involves presenting descriptive statistics, followed by a clear regression analysis. Discussion should interpret results, compare findings with existing literature, and include meaningful tables and graphs.
5. **Conclusion:** Summarize findings, discuss limitations, and suggest future directions.
6. **Referencing:** Requires correct and consistent citations and a well-structured reference list.

Employing a novel dataset, i.e. not employed during the practical sessions, for the assignment will be awarded with a higher grade. For example, see other quantitative dataset from [Secondary datasets for Human Geography](#).

Assessment: How to submit

Normally you don't need to do so, but if something does not work, you may have to install [Quarto](#). Recent versions of RStudio do not require you to do so.

.html file

Submit a single, self-contained .html file. PDF submissions are not accepted.

The .html file is a rendered version of a Quarto Markdown file (`.qmd` file). This will allow you to write a research paper that also includes your working code, without the need of including the data (the code in a `.qmd` file is executed before the document is converted to HTML).

To do so, at the top of your document, include something like this in a YAML cell (see [Template](#), in RStudio, “visual view”, **Insert, YAML block**)

```
---
title: "ENVS225 Assignment"
author: "Anonymous" # do not change
format:
  html:
    self-contained: true
---
```

Once your document is ready with your text and code, render the Quarto document: Click the **Render** button in the RStudio toolbar (blue arrow). This, if everything works (code, existing data, etc), will create an `.html` file in the same location as your `.qmd` file

Assessment Template

Suggested Report Structure — This template follows the requested sections. Each section contains guidance text and an empty R code cell for your analysis. Download it from [here](#), then upload it to the myLabs folder in RStudio (see [Setup](#), section 5) and work on it there.

Start your .qmd file with this YAML header:

```
---
title: "ENVS225 Assignment"
author: "Anonymous"
date: "2026-09-28"
format:
  html:
    self-contained: true
    toc: true
    toc-depth: 2
    code-fold: show
execute:
  warning: false
  message: false
---
```

Setup

Load commonly used libraries for statistical analysis and data manipulation.

```
# Core tidyverse
library(tidyverse) # includes ggplot2, dplyr, tidyr, readr, purrr, tibble, stringr, forcats
```

Warning: package 'ggplot2' was built under R version 4.4.3

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.1.4      v readr      2.1.5
v forcats    1.0.0      v stringr    1.5.1
```

```

v ggplot2 4.0.0      v tibble 3.2.1
v lubridate 1.9.3    v tidyr 1.3.1
v purrr 1.0.2
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag() masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
# Tables & reporting
library(kableExtra)

```

Warning: package 'kableExtra' was built under R version 4.4.2

Attaching package: 'kableExtra'

The following object is masked from 'package:dplyr':

group_rows

```

# Stats helpers
library(broom)

## Already installed in your codespace. On your own PC, install them first.

```

Load your data. If R cannot find the file, see [Setup](#).

```
# df <- read.csv("../data/your_file.csv") # uncomment and change to your file
```

! Important

Where figures and tables go. Put each figure or table in the section that uses it, straight after the paragraph where you first discuss it. **Do not collect them at the end of the report or in an appendix.** Each section below says which figures and tables it needs, if any. Most reports need about **3 tables and 2–3 charts in total**. More charts do not earn more marks: choose each one because it shows something your text discusses (see [Lab: Data Visualisation](#), Parts 3 and 4). Remember that tables count towards the word count.

Once all the chunks execute correctly (no errors when running all the code, Ctrl + Alt + R) you can render the qmd file. Follow the instructions [here](#) to render the document to an HTML file. Remove this message and any irrelevant text from the final submission.

Introduction

Context. Explain why the topic matters (theory, policy, practice), briefly drawing on existing literature.

Aim. State clearly what you want to investigate (e.g., association between X and Y).

Gap & Research Question (RQ). Identify what is missing in the current knowledge and pose 1 RQ (avoid yes/no questions; ask *how, to what extent, which factors*).

Structure. Briefly preview how the rest of the report is organised.

Figures and tables: none. This section is text only.

Literature Review

What we know. Summarise key findings from prior studies relevant to your RQ.

Predictor rationale. Justify the inclusion of variables (theory-driven, prior empirical evidence).

Remaining gap. Specify precisely the gap this report addresses (e.g., population, geography, time period, variable, method).

Note: The goal is a clean, sensible analysis grounded in existing ideas—not necessarily novel theory.

Figures and tables: none. This section is text only.

Methods and Data

Figures and tables in this section: one table (the variables table below). No charts.

Data

Describe the dataset: who collected it, when, whether records are individuals (e.g. FRS, SAR) or geographical units (e.g. Census districts), how many records you use, and any filtering or sampling you applied.

Table 3: Variables used in the analysis. Source: ...

```
# a small table describing your variables (e.g. build a data.frame and show it with kbl());
```

Variables

Name your dependent variable and your independent variables, with their type (numerical or categorical) and units. For categorical variables, state the reference category. Present them in **one short table** (Variable, Description, Type, Units or categories): it describes the variables, it does not report results.

Transformations

Describe any recoding or aggregation (e.g., bin income into bands; reduce age groups from 11 to 3) and justify it. No table or chart needed: if you recode a variable, show its new categories in the variables table above.

Analysis

State which model you use and why, based on your dependent variable: **multiple linear regression** for a numerical dependent variable, **logistic regression** for a binary one.

Important: do not report results here. Descriptive statistics, model output and charts of your data go in Results and Discussion.

Results and Discussion

Figures and tables in this section: one summary table and 2–3 charts (descriptive statistics), then **one model table** (regression model). This is where almost all of your figures and tables belong.

Table 4: Descriptive statistics of the variables in the model. Source: ...

```
# one summary table of all model variables; refer to it as @tbl-descriptives
```

Descriptive statistics

Summarise all the variables in your model in **one table**: mean, standard deviation, median and range for numerical variables; counts and percentages for categorical variables (e.g. `vtable::st()` or `kbl()`, see the Week 5 lab, [Data Visualisation](#)).

Then add **2–3 charts**, each chosen for the variables it shows. Pick from:

- **Distribution of the dependent variable:** histogram (numerical) or bar chart (categorical or binary).
- **Dependent vs a key independent variable:** scatterplot (both numerical), boxplot (categorical vs numerical), or stacked percentage bar chart (both categorical).
- **Associations between independent variables:** correlation matrix or correlogram for numerical variables; Chi-squared test and Cramér's V for categorical variables. This also flags variables that are too closely related to include together.

Comment on what each table and chart shows. Do not overload the report with graphs, and do not show the same result in both a table and a chart. Put each chart in its own code chunk, straight after the paragraph that discusses it.

```
# one chart per chunk; refer to this chart in your text as @fig-dependent
```

Regression model

Present the model in **one table**:

- **Multiple linear regression:** coefficients, standard errors (or 95% confidence intervals), p-values, adjusted R^2 , number of observations.
- **Logistic regression:** odds ratios with 95% confidence intervals, p-values, number of observations.

For each independent variable, report the direction and size of the association in the units of the dependent variable (e.g. percentage points, or odds for logistic regression), whether it is statistically significant, and that it holds with the other variables held constant. Interpret categorical variables against their reference category. Then report the model fit and, in a sentence or two, what the diagnostic checks showed (see the Week 2 lab, [Multiple Linear Regression](#)): describe them in words, you do not need to include the diagnostic plots. The regression model should be the core of this section!

Table 5: Regression model of ... Source: ...

```
# run the model and show a table summarising it (e.g. tidy() + kbl()); refer to it as @tbl-model
```

Discussion

Answer your research question. Compare and contrast your findings with the literature: where do they agree, where do they differ, and why might that be? Remember that regression shows associations, not causes. No new figures or tables here: refer back to the ones above (e.g. “as @tbl-model shows...”).

Captions and labels (all figures and tables): give each one a number and a descriptive title (the `fig-cap` / `tbl-cap` options above do this), label axes with units, state the data source, and refer to each one in the text.

Conclusion

Summary. Recap the main findings vis-à-vis the RQ (do not introduce new results).

Limitations. Offer a brief, honest self-critique (data, measurement, design, external validity).

Implications. Indicate what the findings suggest for practice, policy, or future research.

Figures and tables: none. This section is text only.

References

List every source you cite, in one consistent referencing style (e.g. Harvard). References are not included in the word count.

Samples, Uncertainty and P-values

This page picks up where the Week 1 practical left off. You do not need it to complete any lab, and nothing here is examined on its own. It is here because the ideas below decide whether the numbers in your report actually mean what you say they mean, and because three of them are very commonly got wrong in assignments.

Read it at some point before you write up your results. It should take about twenty minutes.

What you need to have done first: the section *From Sample to Population* in the [Week 1 practical](#), where we established that a sample mean wobbles, and that the size of the wobble is called the standard error.

Setting Up

Everything below uses the same FRS data as the Week 1 practical, so we load it the same way. Check the path — you may need to remove the ...

```
library(dplyr)
library(ggplot2)

frs_data <- read.csv("../data/FRS/FRS16-17_labels.csv")
frs_independent <- frs_data %>% filter(dependent != "Dependent")
```

A note before we start: none of the code below will give you exactly the numbers printed on this page, because every chunk draws a fresh random sample. That is the entire point of the page. Your figures should be *close* to these, not identical, and if you run a chunk twice you will get two different answers.

Why Sample Size Matters

In the practical we found that samples of 500 people missed the true mean income by about £1,200 on average, and that this figure — the standard error — can be calculated as the standard deviation divided by the square root of the sample size.

That square root has a practical consequence. Let's repeat the exercise with smaller and larger samples:

```
means_100 <- replicate(1000, mean(sample(frs_independent$income_gross, size = 100)))
means_2000 <- replicate(1000, mean(sample(frs_independent$income_gross, size = 2000)))

sd(means_100) # standard error with n = 100
```

```
[1] 2718.75
```

```
sd(means_2000) # standard error with n = 2000
```

```
[1] 575.8038
```

Roughly £2,800 for samples of 100, against £600 for samples of 2000. Bigger samples give more precise answers — but look at the *rate*. To halve your uncertainty you must **quadruple** your sample.

This is why national surveys are expensive. It is also why you should be sceptical of confident claims made about small subgroups: a survey of 40,000 people may contain only 90 people in the particular category you want to talk about, and the uncertainty for that group is set by the 90, not the 40,000.

Worth doing: find a subgroup in the FRS that interests you — one region, say, or one age group. How many people are in it? Using the formula above, how precisely can you estimate its mean income? If the answer is “not very”, that is a limitation worth stating in your report.

Confidence Intervals

Reporting “£22,544” on its own pretends to a precision we do not have. A **confidence interval** reports the estimate together with its uncertainty, and R will build one for us:

```
one_sample <- sample(frs_independent$income_gross, size = 500)

t.test(one_sample)
```

One Sample t-test

```
data: one_sample
t = 20.265, df = 499, p-value < 2.2e-16
```

```
alternative hypothesis: true mean is not equal to 0
95 percent confidence interval:
 21545.81 26172.10
sample estimates:
mean of x
 23858.95
```

Ignore most of that output and look at the part labelled **95 percent confidence interval**.

The interval is, roughly, “our estimate, give or take about two standard errors”. Its meaning is best stated as a statement about the *method*: if we repeated this whole exercise many times, about 95 out of every 100 intervals built this way would contain the true population mean.

A confidence interval is almost always more informative than a single number, and reporting one in your assignment is a mark of careful work.

Seeing What “95%” Means

That definition is a mouthful, and it is far easier to believe once you have watched it happen. So let’s build not one confidence interval, but a hundred.

The idea is the same as before: take a sample of 500 people, build a confidence interval from it, write it down, and repeat another 99 times. Because we are pretending the FRS is the whole population, we know the true answer, so afterwards we can simply count how many of our intervals caught it.

```
# Take 100 samples of 500 people, and keep the confidence interval from each one
intervals <- replicate(100, t.test(sample(frs_independent$income_gross, 500))$conf.int)
```

`t.test(...)$conf.int` runs the test but keeps *only* the confidence interval, throwing the rest of the output away. The `$` picks one piece out of a result, just as `Grades$Name` picked one column out of a table in the practical.

The result, `intervals`, has two rows and 100 columns: one column per sample, with its lower limit on the first row and its upper limit on the second. We can pull those two rows out to build an ordinary table:

```
many_intervals <- data.frame(
  sample_number = 1:100,
  lower = intervals[1, ], # all of row 1: the lower limits
  upper = intervals[2, ] # all of row 2: the upper limits
)
```

```
head(many_intervals)
```

	sample_number	lower	upper
1	1	19530.99	23276.03
2	2	22025.34	27397.54
3	3	19275.32	22771.04
4	4	20781.39	25808.32
5	5	21542.95	26774.41
6	6	18593.46	24204.00

Now work out the true answer, and check each interval against it:

```
true_mean <- mean(frs_independent$income_gross)

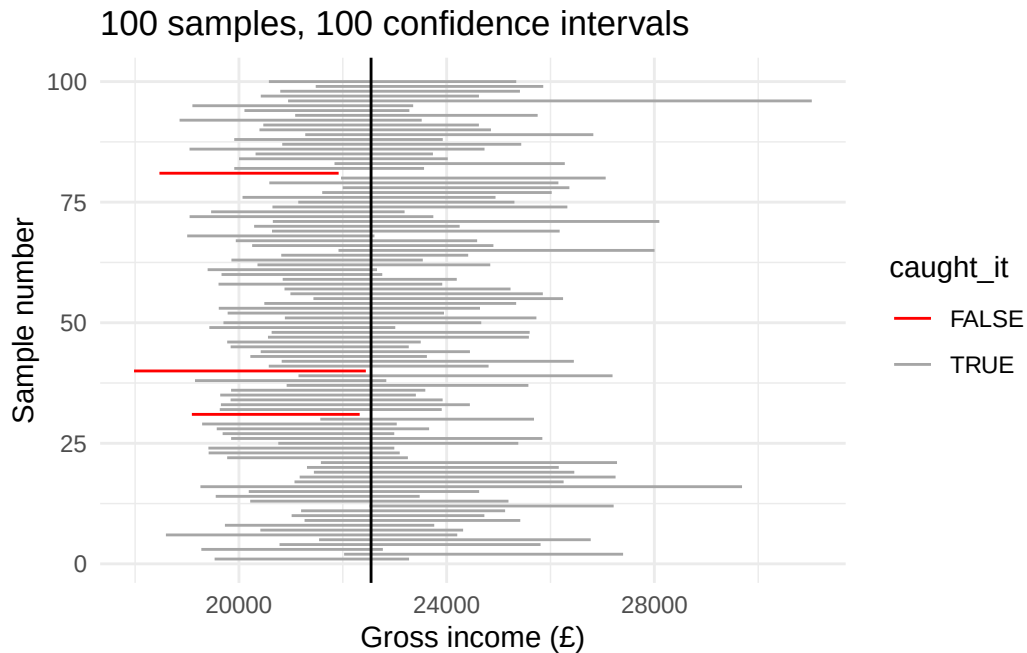
# TRUE if the interval caught the true mean, FALSE if it missed
many_intervals$caught_it <- many_intervals$lower < true_mean & many_intervals$upper > true_m

table(many_intervals$caught_it)
```

FALSE	TRUE
3	97

A handful missed — usually somewhere between about three and ten of the hundred. Let's draw them. Each horizontal line is one sample's confidence interval, and the vertical line is the true mean income of the population:

```
ggplot(many_intervals, aes(y = sample_number, colour = caught_it)) +
  geom_linerange(aes(xmin = lower, xmax = upper)) +
  geom_vline(xintercept = true_mean) +
  scale_colour_manual(values = c("red", "grey65")) +
  labs(
    title = "100 samples, 100 confidence intervals",
    x = "Gross income (£)",
    y = "Sample number"
  ) +
  theme_minimal()
```



This picture *is* the definition of a confidence interval:

- Every interval is a different width and sits in a different place, because every sample was different.
- Most straddle the true mean. A few — the red ones — do not.
- Nothing is wrong with the red intervals. They were built in exactly the same way as the others. They were simply unlucky samples.

And here is the part that matters for your report. **In real research you only ever get one of these lines.** You never find out whether yours is grey or red, because you never learn the true answer. What “95% confident” buys you is not certainty about your own interval; it is the knowledge that the *procedure* you used lands on the truth about 19 times out of 20.

Worth doing: change 500 to 2000 in the code above and run the whole thing again.

You should find the intervals roughly halve in width, because we quadrupled the sample. But the number of red ones does **not** shrink in the same way — it stays somewhere around five, bouncing about from run to run because that count is itself a random quantity.

This is the point. Bigger samples buy you *precision*: a narrower interval, a sharper answer. They do not buy you *reliability*: the 95% was fixed by us when we asked for a 95% interval, and it stays at 95% whatever the sample size.

One honest footnote. If you count carefully you will often find rather more than five red intervals. That is not a mistake either. Incomes are so badly skewed — remember the handful of people earning over a million — that confidence intervals built this way come out slightly

too narrow, and catch the truth about 93% of the time rather than 95%. This is a small example of a large lesson: statistical tools come with assumptions attached, and skewed data strains them.

Significance and Sample Size

In the practical we found a gender income gap of about £9,300 across 33,836 people, with a p-value too small for R to print. Now watch what happens when we ask the identical question of only 60 people:

```
small_sample <- frs_independent %>% slice_sample(n = 60)
t.test(income_gross ~ sex, data = small_sample)
```

Welch Two Sample t-test

```
data: income_gross by sex
t = -0.24322, df = 49.56, p-value = 0.8088
alternative hypothesis: true difference in means between group Female and group Male is not equal to 0
95 percent confidence interval:
 -12421.288  9738.464
sample estimates:
mean in group Female    mean in group Male
      25928.00         27269.41
```

Worth doing: run those two lines four or five times over, and watch the p-value each time.

You will see it jump around wildly. Sometimes the gap comes out statistically significant; most of the time it does not — in fact, with 60 people this test only reaches $p < 0.05$ about a third of the time. The confidence interval for the difference will usually include zero, which is the same message in a different form.

Nothing about the UK changed between the big test and the small one. There really is a gender income gap. Only our ability to *detect* it changed.

This is the lesson to take away: “not statistically significant” means “*this sample cannot settle the question*”. It does **not** mean “there is no difference”.

In your report, this is a matter of wording, and markers notice. “No significant association was found” is defensible. “There is no association” is not, and it is a much stronger claim than your data can support.

What a P-value Is Not

Three misreadings are extremely common in student reports, and each one costs marks:

- A p-value is **not** the probability that the null hypothesis is true, and it is **not** the probability that your finding is correct. It is calculated *assuming* the null hypothesis is true, so it cannot tell you how likely that assumption is. Three asterisks do not mean “99.9% certain there is a relationship”.
- A small p-value does **not** mean the effect is large or important. With a big enough sample, a trivially small effect will come out “highly significant”. Significance tells you an effect is *detectable*; only the coefficient itself tells you whether it *matters*. Always report and interpret both.
- A p-value says nothing at all about **causation**. It cannot distinguish a real causal relationship from a confounded one. That judgement comes from your literature review and your research design, never from the output.

A Note on Census Data

Everything on this page assumes your data are a *random sample* of some larger population. That is true of the FRS, the APS and the SAR.

It is not true of the Census. When you run a regression on the 2021 Census district data in Week 2, you are working with every district in England and Wales, not a sample of them. Strictly speaking there is no sampling uncertainty to quantify, because there was no sampling.

Statisticians handle this by treating the observed pattern as one realisation of a social process that could have turned out differently — the districts we see are one roll of the dice, and the p-value asks how easily that roll could have come up by chance. That is a reasonable way to reason, but it rests on a weaker footing than a genuine random sample, and it is a judgement rather than a fact about the data.

You do not need to resolve this. But if you use Census data in your assignment, a sentence acknowledging it in your limitations section is exactly the kind of critical awareness the higher marking bands reward.

1 Lab: Introduction to R

The following material has been readapted from:

- <https://dereksonderegger.github.io/570L/1-introduction.html> by Derek L. Sonderegger;

Open this lab in RStudio: in the Files pane, click labs and then 00.introR.qmd. Save your own copy in myLabs (File -> Save As...) and work on the copy. Work through it from top to bottom, running each code chunk. (First time? Follow the [Setup](#) guide first.)

This short lab shows you how to find your way around RStudio and how to run R code. It takes about 20 minutes. Then move on to [Lab: Exploring a Dataset](#), the main practical for Week 1.

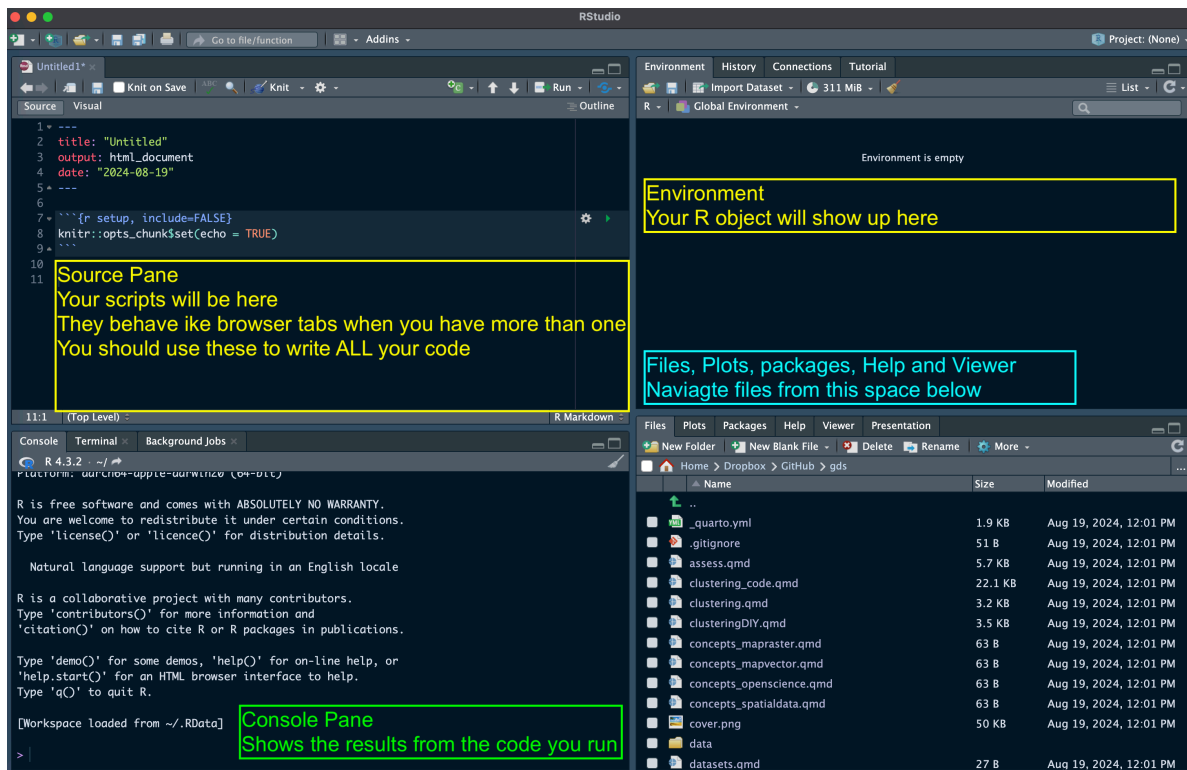
1.1 R and RStudio

R is a free program for statistics and data analysis. You use it by writing code, not by clicking menus. This takes a little getting used to, but it has a big advantage: your code is an exact record of every step of your analysis, so you (or anyone else) can run it again and get the same result.

RStudio is the program you use to write and run R code. In this module it runs in your browser, and R and all the packages you need are already installed.

1.1.1 The four panels

RStudio looks something like this:



- **Top left, Source:** the file you are working on, for example this lab.
- **Bottom left, Console:** where R runs code and prints the results. You can also type code here directly and press Enter. Ignore the **Terminal** tab next to it: you don't need it.
- **Top right, Environment:** the data and variables you have created so far.
- **Bottom right, Files / Plots / Packages / Help:** your course folder, your charts, and help pages.

You can change the colours and font size in Tools -> Global Options -> Appearance.

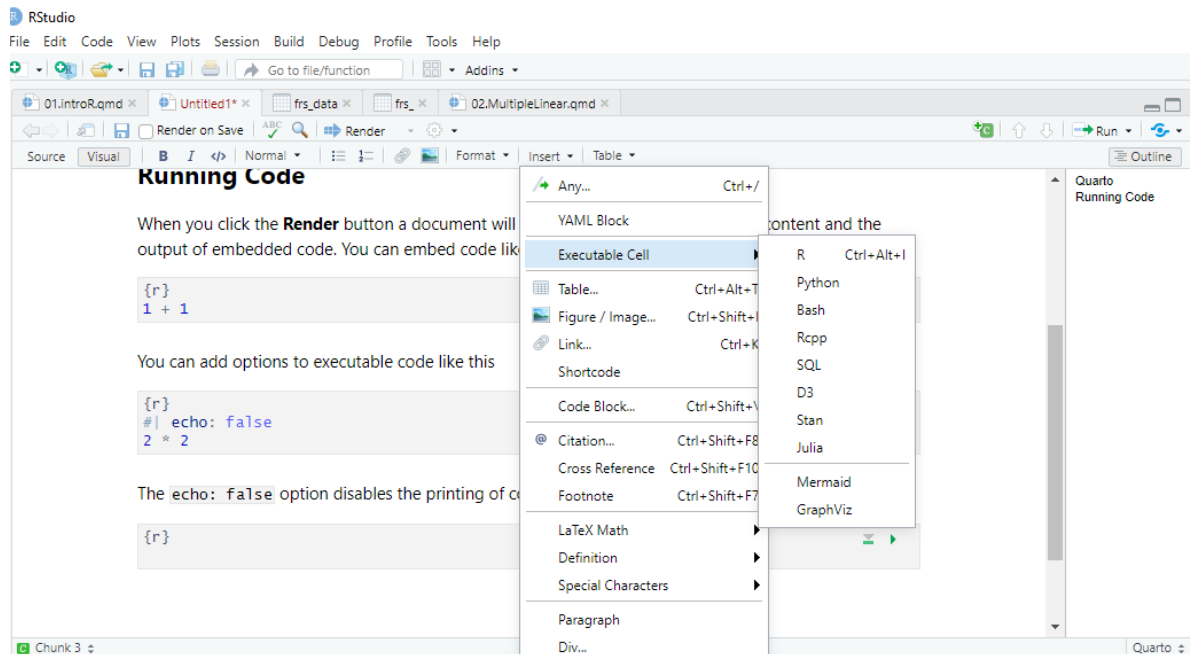
1.2 Quarto documents and code chunks

The labs are **Quarto documents** (.qmd files). A Quarto document mixes normal text, like this paragraph, with **code chunks**: the grey boxes that contain R code. You will write your assessment report in the same kind of file.

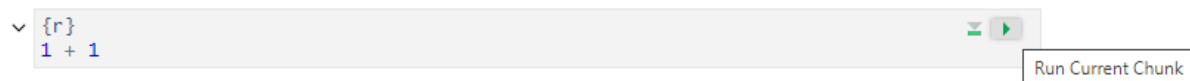
A code chunk starts with ````{r}` and ends with `````:



The buttons **Source** and **Visual** at the top left of the file switch between two views of the same file. In the **Visual** view you can also insert a chunk from **Insert -> Executable Cell -> R**:



To run a chunk, click the green arrow in its top-right corner. The result appears under the chunk.



Task: Run the chunk below.

```
print("hello world")
```

```
[1] "hello world"
```

Some useful shortcuts (use Cmd instead of Ctrl on a Mac):

Shortcut	What it does
Ctrl+Enter	Run the line where your cursor is
Ctrl+Shift+Enter	Run the whole chunk
Ctrl+Alt+I	Insert a new chunk (or use Code -> Insert Chunk)
Ctrl+S	Save the file

To add your own notes, just type in the white space between chunks. To try out your own code, insert a new chunk.

1.3 R as a calculator

Task: Run each chunk in this section and check that you understand the result.

```
# Some simple addition. Text after a # is a comment: R ignores it.
2+3
```

```
[1] 5
```

```
6*8
```

```
[1] 48
```

```
4^3
```

```
[1] 64
```

```
exp(1) # exp() is the exponential function
```

```
[1] 2.718282
```

R has most constants and mathematical functions you could want. For example, **abs()** gives the absolute value of a number, and **round()** rounds it to the nearest whole number.

```
pi # the constant 3.14159265...
```

```
[1] 3.141593
```

```
abs(-1.77)
```

```
[1] 1.77
```

```
round(1.77)
```

```
[1] 2
```

1.3.1 Functions and arguments

`abs()` and `round()` are **functions**. The values you give a function inside the brackets are its **arguments**, separated by commas. Some arguments are required and some are optional.

The `log()` function calculates a logarithm. It takes the number `x` and the `base`. If you name the arguments, the order doesn't matter:

```
log(x=5, base=10)
```

```
[1] 0.69897
```

```
log(base=10, x=5)
```

```
[1] 0.69897
```

If you don't name them, R assumes that the first value is `x` and the second is `base`:

```
log(5, 10)
```

```
[1] 0.69897
```

```
log(10, 5)
```

```
[1] 1.430677
```

1.3.2 Getting help

To see what a function does and which arguments it takes, type `?` followed by its name in the console, for example `?log`. The help page opens in the **Help** tab, bottom right.

1.4 Variables

To use a result later, save it in a **variable**. R uses the arrow `<-` for this (`=` also works, but pick one and stick with it).

```
var <- 2*7.5      # create two variables
another_var <- 5  # notice they appear in the Environment panel
var
```

```
[1] 15
```

```
var * another_var
```

```
[1] 75
```

Variable names cannot start with a number or contain spaces, and they are case sensitive: `var` and `Var` are two different variables.

1.5 Packages

Packages add extra functions to R. For example, `dplyr` has functions for working with data and `ggplot2` makes charts. **All the packages you need in this module are already installed in your codespace.** You just need to load a package with `library()` before you use it, once per session:

```
library(dplyr)  # load the dplyr package; we will use it in the next lab
```

Note

If you use RStudio on a university PC or your own laptop instead of the codespace, a package may be missing. You will see an error like **there is no package called 'dplyr'**. Install the package once with **Tools -> Install Packages...**, then run `library()` again. See [Setup: RStudio on a PC or Laptop](#) to install all the course packages at once.

1.6 Next

That's all you need to start. Save this file (Ctrl+S), then open `01.DataExploration.qmd` in the `labs` folder and save your copy in `myLabs`: [Lab: Exploring a Dataset](#).

2 Lab: Exploring a Dataset

The following material has been readapted from:

- <https://pietrostefani.github.io/gds/envIRONR.html> by Elisabetta Pietrostefani and Carmen Cabrera-Arnau
- https://raw.githubusercontent.com/dereksonderegger/570L/master/07_DataImport.Rmd

The lecture's slides can be found [here](#).

Before completing the practical, please take this [quiz](#).

Open this lab in RStudio: in the Files pane, click labs and then `01.DataExploration.qmd`. Save your own copy in myLabs (File -> Save As...) and work on the copy. Work through it from top to bottom, running each code chunk and adding your notes and answers in the file. (First time? See the [Setup](#) guide, then do [Lab: Introduction to R](#), which shows how to run code chunks.)

Follow the practical below. You can describe what you are doing in normal text. See [here](#) for how to format normal text in Markdown documents.

2.1 Practice: Dataset and Dataframes

Within this module we will be working with data stored in so-called datasets. A dataset is a structured collection of data points that represent various measurements or observations, often organized in a tabular format with rows and columns. A dataset might contain information about different locations, such as neighborhoods or cities, with each row representing a place and each column detailing characteristics like population density, average income, or number of green parks. For example, a dataset could be compiled to study patterns in urban mobility, where the data includes the number of daily commuters, the distance they travel, and the mode of transport they use. Datasets provide the essential building blocks for statistical analysis; they enable exploring relationships, identifying patterns, and drawing conclusions about certain phenomena.

Examples of everyday datasets:

- **Premier League Standings:** Each row represents a team, with columns for points, games played, wins, draws, and losses.

- **Movie Dataset:** Each row represents a movie, with columns showing its title, genre, release year, director, and rating.
- **Weather Dataset:** Each row shows a day's weather in a city, with columns for temperature, humidity, wind speed, and precipitation.

Usually, data is organized in

- **Columns** of data representing some trait or variable that we might be interested in. In general, we might wish to investigate the relationship between variables.
- **Rows** represent a single object on which the column traits are measured.

For example, in a grade book for recording students scores throughout the semester, there is one row for every student and columns for each assignment. A greenhouse experiment dataset will have a row for every plant and columns for treatment type and biomass.

2.1.1 Datasets in R

In R, we want a way of storing data where it feels just as if we had an Excel Spreadsheet where each row represents an observation and each column represents some information about that observation. We will call this object a `data.frame`, an R representation of a data set. The easiest way to understand data frames is to create one.

Task: Run the chunk below. It creates a `data.frame` that represents an instructor's grade book, where each row is a student, and each column represents some sort of assessment.

```
library(dplyr)

Grades <- data.frame(
  Name = c('Bob', 'Jeff', 'Mary', 'Valerie'),
  Exam.1 = c(90, 75, 92, 85),
  Exam.2 = c(87, 71, 95, 81)
)
# Show the data.frame
# View(Grades) # show the data in an Excel-like tab. Doesn't work when knitting
Grades          # show the output in the console. This works when knitting
```

	Name	Exam.1	Exam.2
1	Bob	90	87
2	Jeff	75	71
3	Mary	92	95
4	Valerie	85	81

R allows two different ways to access elements of the `data.frame`. First is a matrix-like notation for accessing particular values.

Format	Result
<code>[a,b]</code>	Element in row <code>a</code> and column <code>b</code>
<code>[a,]</code>	All of row <code>a</code>
<code>[,b]</code>	All of column <code>b</code>

Because the columns have meaning and we have given them column names, it is desirable to want to access an element by the name of the column as opposed to the column number.

Task: Run:

```
Grades[, 2]      # print out all of column 2
```

```
[1] 90 75 92 85
```

```
Grades$Name      # The $-sign means to reference a column by its label
```

```
[1] "Bob"      "Jeff"     "Mary"     "Valerie"
```

2.1.2 Importing Data in R

Usually we won't type the data in by hand, but rather load the data from some package. Reading data from external sources is a necessary skill.

Comma Separated Values Data

To consider how data might be stored, we first consider the simplest file format: the comma separated values file (`.csv`). In this file type, each of the “cells” of data are separated by a comma. For example, the data file storing scores for three students might be as follows:

```
Able, Dave, 98, 92, 94
Bowles, Jason, 85, 89, 91
Carr, Jasmine, 81, 96, 97
```

Typically when you open up such a file on a computer with MS Excel installed, Excel will open up the file assuming it is a spreadsheet and put each element in its own cell. However, you can also open the file using a more primitive program (say Notepad in Windows, TextEdit on a Mac) you'll see the raw form of the data.

Having just the raw data without any sort of column header is problematic (which of the three exams was the final??). Ideally we would have column headers that store the name of the column.

```
LastName, FirstName, Exam1, Exam2, FinalExam
Able, Dave, 98, 92, 94
Bowles, Jason, 85, 89, 91
Carr, Jasmine, 81, 96, 97
```

Reading (.csv) files

To make R read in the data arranged in this format, we need to tell R three things:

1. Where does the data live? Often this will be the name of a file on your computer, but the file could just as easily live on the internet (provided your computer has internet access).
2. Is the first row data or is it the column names?
3. What character separates the data? Some programs store data using tabs to distinguish between elements, some others use white space. R's mechanism for reading in data is flexible enough to allow you to specify what the separator is.

The primary function that we'll use to read data from a file and into R is the function `read.csv()`. This function has many optional arguments but the most commonly used ones are outlined in the table below.

Argument	Default	Description
<code>file</code>	Required	A character string denoting the file location.
<code>header</code>	TRUE	Specifies whether the first line contains column headers.
<code>sep</code>	","	Specifies the character that separates columns. For <code>read.csv()</code> , this is usually a comma.

Argument	Default	Description
<code>skip</code>	0	The number of lines to skip before reading data; useful for files with descriptive text before the actual data.
<code>na.strings</code>	"NA"	Values that represent missing data; multiple values can be specified, e.g., <code>c("NA", "-9999")</code> .
<code>quote</code>	"	Specifies the character used to quote character strings, typically " or '.
<code>stringsAsFactors</code>	FALSE	Controls whether character strings are converted to factors; FALSE means they remain as character data.
<code>row.names</code>	NULL	Allows specifying a column as row names, or assigning NULL to use default indexing for rows.
<code>colClasses</code>	NULL	Specifies the data type for each column to speed up reading for large files, e.g., <code>c("character", "numeric")</code> .
<code>encoding</code>	"unknown"	Sets the text encoding of the file, which can be useful for files with special or international characters.

Most of the time you just need to specify the file.

Task: Let's read in a dataset of terrorist attacks that have taken place in the UK:

```
attacks <- read.csv(file = '../data/attacksUK.csv') # where the data lives
View(attacks)
```

2.2 Practice: Descriptive Statistics

2.2.1 Summarizing Data

It is very important to be able to take a data set and produce summary statistics such as the mean and standard deviation of a column. For this sort of manipulation, we use the package `dplyr`. This package allows chaining together many common actions to form a particular task.

The fundamental operations to perform on a data set are:

- **Subsetting** - Returns a dataframe with only particular columns or rows
 - `select` - Selecting a subset of columns by name or column number.
 - `filter` - Selecting a subset of rows from a data frame based on logical expressions.
 - `slice` - Selecting a subset of rows by row number.
- `arrange` - Re-ordering the rows of a data frame.
- `mutate` - Add a new column that is some function of other columns.
- `summarise` - calculate some summary statistic of a column of data. This collapses a set of rows into a single row.

Each of these operations is a function in the package `dplyr`. These functions all have a similar calling syntax:

- The first argument is a data set.
- Subsequent arguments describe what to do with the input data frame and you can refer to the columns without using the `df$column` notation.

All of these functions will return a data set.

Let's consider the `summarise` function to calculate the mean score for `Exam.1`. Notice that this takes a data frame of four rows, and summarizes it down to just one row that represents the summarized data for all four students.

```
library(dplyr) # load the library
Grades %>%
  summarize( Exam.1.mean = mean( Exam.1 ) )
```

```
Exam.1.mean
1          85.5
```

Similarly you could calculate the **standard deviation** for the exam as well.

```
Grades %>%  
  summarize( Exam.1.mean = mean( Exam.1 ),  
             Exam.1.sd   = sd(   Exam.1   ) )
```

```
Exam.1.mean Exam.1.sd  
1          85.5  7.593857
```

Task: Insert a new chunk below and calculate the mean and standard deviation of Exam.2. Type the code yourself instead of copying it: it is the best way to learn it.

The %>% operator works by translating the command `a %>% f(b)` to the expression `f(a,b)`. This operator works on any function `f`. This is useful when we want to start with `x`, and first apply a function `f()`, then `g()`, and then `h()`; the usual R command would be `h(g(f(x)))` which is hard to read. Using the pipe command %>%, this sequence of operations becomes `x %>% f() %>% g() %>% h()`.

Below, the code takes the Grades dataframe and calculates a column for the average exam score, and then sorts the data according to that average score

```
Grades %>% mutate( Avg.Score = (Exam.1 + Exam.2) / 2 ) %>% arrange( Avg.Score )
```

	Name	Exam.1	Exam.2	Avg.Score
1	Jeff	75	71	73.0
2	Valerie	85	81	83.0
3	Bob	90	87	88.5
4	Mary	92	95	93.5

You don't have to memorise this.

Let's go back to the terrorist attacks dataframe. There are attacks perpetrated by several different groups. Each record is a single attack and contains information about who perpetrated the attack, what year, how many were killed and how many were wounded. You can get a glimpse of the dataframe with the function `head`

```
head(attacks, n = 10)
```

	nrKilled	nrWound	year	country	group
1	0	0	2005	United Kingdom	Abu Hafs al-Masri Brigades
2	0	0	2005	United Kingdom	Abu Hafs al-Masri Brigades
3	0	0	2005	United Kingdom	Abu Hafs al-Masri Brigades

4	0	0	2005	United Kingdom	Abu Hafs al-Masri Brigades
5	0	1	1982	United Kingdom	Abu Nidal Organization (ANO)
6	0	0	2014	United Kingdom	Anarchists
7	0	0	2014	United Kingdom	Anarchists
8	0	0	2014	United Kingdom	Anarchists
9	0	0	2014	United Kingdom	Anarchists
10	0	0	2014	United Kingdom	Anarchists
				attack	target
1				Bombing/Explosion	Transportation
2				Bombing/Explosion	Transportation
3				Bombing/Explosion	Transportation
4				Bombing/Explosion	Transportation
5				Assassination	Government (Diplomatic)
6				Facility/Infrastructure Attack	Business
7				Facility/Infrastructure Attack	Business
8				Facility/Infrastructure Attack	Business
9				Facility/Infrastructure Attack	Private Citizens & Property
10				Facility/Infrastructure Attack	Police
				weapon	
1				Explosives/Bombs/Dynamite	
2				Explosives/Bombs/Dynamite	
3				Explosives/Bombs/Dynamite	
4				Explosives/Bombs/Dynamite	
5				Firearms	
6				Incendiary	
7				Incendiary	
8				Incendiary	
9				Incendiary	
10				Incendiary	

We might want to compare different actors and see the mean and standard deviation of the number of people wounded, by each group's attack, across time. To do this, we are still going to use `summarise()`, but we will precede it with `group_by(group)` to tell the subsequent `dplyr` functions to perform the actions separately for each group.

```
attacks %>%
  group_by( group) %>%
  summarise( Mean = mean(nrWound),
             Std.Dev = sd(nrWound))
```

```
# A tibble: 38 x 3
  group
```

```
Mean Std.Dev
```

	<chr>	<dbl>	<dbl>
1	Abu Hafs al-Masri Brigades	0	0
2	Abu Nidal Organization (ANO)	1	NA
3	Anarchists	0	0
4	Animal Liberation Front (ALF)	0.0833	0.282
5	Animal Rights Activists	0.167	0.389
6	Armenian Secret Army for the Liberation of Armenia	0.2	0.447
7	Black September	0.667	0.577
8	Continuity Irish Republican Army (CIRA)	0.7	3.12
9	Dissident Republicans	0.0758	0.267
10	Informal Anarchist Federation	0	0

i 28 more rows

Task: Run the chunk above. Then insert a new chunk and try out another categorical variable instead of `group` (e.g. `year`) and `nrKilled` instead of `nrWound`.

Let's now move to another dataset to address a research question.

For illustration purposes, we will use the **Family Resources Survey (FRS)**. The FRS is an annual survey conducted by the UK government that collects detailed information about the income, living conditions, and resources of private households across the United Kingdom. Managed by the Department for Work and Pensions (DWP), the FRS provides data that is essential for understanding the economic and social conditions of households and informing public policy.

Consider questions such as:

- How many respondents (persons) are there in the 2016-17 FRS?
- How many variables (population attributes) are there?
- What types of variables are present in the FRS?
- What is the most detailed geography available in the FRS?

Task: To answer these questions, download `FRS16-17_labels.csv` from the [FRS page on Canvas](#) and upload it to a new folder `data/FRS` in RStudio (see [Setup](#), section 5). Then load and inspect the dataset.

```
# the FRS dataset should be already loaded, otherwise
frs_data <- read.csv("../data/FRS/FRS16-17_labels.csv")

# Display basic structure
glimpse(frs_data)
```

```
Rows: 44,145
Columns: 45
```

\$ household	<int> 6087, 6101, 6103, 6122, 6134, 6136, 6138, 6140, 6143,~
\$ family	<int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,~
\$ person	<int> 5, 3, 3, 3, 2, 4, 4, 3, 3, 4, 4, 3, 4, 2, 5, 3, 4, 3,~
\$ country	<chr> "England", "England", "England", "Northern Ireland", ~
\$ region	<chr> "London", "South East", "Yorks and the Humber", "Nort~
\$ age_group	<chr> "05-10", "05-10", "05-10", "05-10", "05-10", "05-10",~
\$ sex	<chr> "Female", "Male", "Male", "Female", "Female", "Female~
\$ marital_status	<chr> "Single", "Single", "Single", "Single", "Single", "Si~
\$ ethnicity	<chr> "Mixed / multiple ethnic groups", "White", "White", "~
\$ hrp	<chr> "Not HRP", "Not HRP", "Not HRP", "Not HRP", "Not HRP"~
\$ rel_to_hrp	<chr> "Son/daughter (incl. adopted)", "Son/daughter (incl. ~
\$ lifestage	<chr> "Child (0-17)", "Child (0-17)", "Child (0-17)", "Chil~
\$ dependent	<chr> "Dependent", "Dependent", "Dependent", "Dependent", "~
\$ arrival_year	<chr> "UK Born", "UK Born", "UK Born", "UK Born", "UK Born"~
\$ birth_country	<chr> "Dependent child", "Dependent child", "Dependent chil~
\$ care_hours	<chr> "0 hours per week", "0 hours per week", "0 hours per ~
\$ educ_age	<chr> "Dependent child", "Dependent child", "Dependent chil~
\$ educ_type	<chr> "School (full-time)", "School (full-time)", "School (~
\$ fam_youngest	<chr> "7", "4", "0", "7", "0", "9", "10", "0", "3", "10", "~
\$ fam_toddlers	<int> 0, 1, 1, 0, 2, 0, 0, 2, 1, 0, 0, 1, 0, 1, 0, 0, 0, 1,~
\$ fam_size	<int> 4, 4, 4, 3, 4, 4, 3, 5, 4, 4, 3, 4, 4, 3, 5, 4, 4, 4,~
\$ happy	<chr> "Dependent child", "Dependent child", "Dependent chil~
\$ health	<chr> "Not known", "Not known", "Not known", "Not known", "~
\$ hh_accom_type	<chr> "Terraced house/bungalow", "Detached house/bungalow",~
\$ hh_benefits	<int> 10868, 0, 1768, 8632, 8372, 1768, 1768, 1768, 0, 0, 1~
\$ hh_composition	<chr> "Three or more adults, 1+ children", "One adult femal~
\$ hh_ctax_band	<chr> "Band D", "Band F", "Band A", "Band B", "Band A", "Ba~
\$ hh_housing_costs	<chr> "4316", "10296", "5408", "Northern Ireland", "5720", ~
\$ hh_income_gross	<int> 54236, 180804, 26936, 19968, 17992, 76596, 31564, 366~
\$ hh_income_net	<int> 44668, 120640, 23556, 19968, 17992, 62868, 29744, 287~
\$ hh_size	<int> 5, 4, 4, 3, 4, 4, 4, 5, 4, 4, 4, 4, 4, 5, 5, 4, 4, 4,~
\$ hh_tenure	<chr> "Mortgaged (including part rent / part own)", "Mortga~
\$ highest_qual	<chr> "Dependent child", "Dependent child", "Dependent chil~
\$ income_gross	<int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
\$ income_net	<int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
\$ jobs	<chr> "Dependent child", "Dependent child", "Dependent chil~
\$ life_satisf	<chr> "Dependent child", "Dependent child", "Dependent chil~
\$ nssec	<chr> "Dependent child", "Dependent child", "Dependent chil~
\$ sic_chapter	<chr> "Dependent child", "Dependent child", "Dependent chil~
\$ sic_division	<chr> "Dependent child", "Dependent child", "Dependent chil~
\$ soc2010	<chr> "Dependent child", "Dependent child", "Dependent chil~
\$ work_hours	<chr> "Dependent child", "Dependent child", "Dependent chil~
\$ workstatus	<chr> "Dependent Child", "Dependent Child", "Dependent Chil~

```
$ years_ft_work    <chr> "Dependent child", "Dependent child", "Dependent chil~
$ survey_weight    <int> 2315, 1317, 2449, 427, 1017, 1753, 1363, 1344, 828, 1~
```

and summary:

```
summary(frs_data)
```

household	family	person	country
Min. : 1	Min. :1.000	Min. :1.00	Length:44145
1st Qu.: 4816	1st Qu.:1.000	1st Qu.:1.00	Class :character
Median : 9673	Median :1.000	Median :2.00	Mode :character
Mean : 9677	Mean :1.106	Mean :1.98	
3rd Qu.:14553	3rd Qu.:1.000	3rd Qu.:3.00	
Max. :19380	Max. :6.000	Max. :9.00	
region	age_group	sex	marital_status
Length:44145	Length:44145	Length:44145	Length:44145
Class :character	Class :character	Class :character	Class :character
Mode :character	Mode :character	Mode :character	Mode :character
ethnicity	hrp	rel_to_hrp	lifestage
Length:44145	Length:44145	Length:44145	Length:44145
Class :character	Class :character	Class :character	Class :character
Mode :character	Mode :character	Mode :character	Mode :character
dependent	arrival_year	birth_country	care_hours
Length:44145	Length:44145	Length:44145	Length:44145
Class :character	Class :character	Class :character	Class :character
Mode :character	Mode :character	Mode :character	Mode :character
educ_age	educ_type	fam_youngest	fam_toddlers
Length:44145	Length:44145	Length:44145	Min. :0.0000
Class :character	Class :character	Class :character	1st Qu.:0.0000
Mode :character	Mode :character	Mode :character	Median :0.0000
			Mean :0.2557
			3rd Qu.:0.0000
			Max. :4.0000

fam_size	happy	health	hh_accom_type
Min. :1.000	Length:44145	Length:44145	Length:44145
1st Qu.:2.000	Class :character	Class :character	Class :character
Median :2.000	Mode :character	Mode :character	Mode :character
Mean :2.599			
3rd Qu.:4.000			
Max. :9.000			
hh_benefits	hh_composition	hh_ctax_band	hh_housing_costs
Min. : 0	Length:44145	Length:44145	Length:44145
1st Qu.: 0	Class :character	Class :character	Class :character
Median : 1768	Mode :character	Mode :character	Mode :character
Mean : 5670			
3rd Qu.:10192			
Max. :54080			
hh_income_gross	hh_income_net	hh_size	hh_tenure
Min. : -326092	Min. : -334776	Min. : 1.00	Length:44145
1st Qu.: 22256	1st Qu.: 20748	1st Qu.:2.00	Class :character
Median : 35984	Median : 31512	Median :3.00	Mode :character
Mean : 46076	Mean : 37447	Mean :2.96	
3rd Qu.: 57252	3rd Qu.: 47008	3rd Qu.:4.00	
Max. :1165216	Max. :1116596	Max. :9.00	
highest_qual	income_gross	income_net	jobs
Length:44145	Min. : -354848	Min. : -358592	Length:44145
Class :character	1st Qu.: 52	1st Qu.: 0	Class :character
Mode :character	Median : 12740	Median : 12012	Mode :character
	Mean : 17305	Mean : 14204	
	3rd Qu.: 23712	3rd Qu.: 20384	
	Max. :1127360	Max. :1110928	
life_satisf	nssec	sic_chapter	sic_division
Length:44145	Length:44145	Length:44145	Length:44145
Class :character	Class :character	Class :character	Class :character
Mode :character	Mode :character	Mode :character	Mode :character
soc2010	work_hours	workstatus	years_ft_work
Length:44145	Length:44145	Length:44145	Length:44145
Class :character	Class :character	Class :character	Class :character
Mode :character	Mode :character	Mode :character	Mode :character

survey_weight

Min. : 221
 1st Qu.: 1097
 Median : 1380
 Mean : 1459
 3rd Qu.: 1742
 Max. : 39675

2.2.2 Understanding the Structure of the FRS Datafile

In the FRS data structure, each row represents a person, but:

- Each person is nested within a family.
- Each family is nested within a household.

Below is an example dataset structure:

household	family	person	region	age_group	sex	marital_status	rel_to_hrp
1	1	1	London	40-44	Female	Married/Civil partner-ship	Spouse
1	1	2	London	40-44	Male	Married/Civil partner-ship	Household Representative
1	1	3	London	5-10	Male	Single	Son/daughter (incl. adopted)
1	1	4	London	5-10	Female	Single	Son/daughter (incl. adopted)
1	1	5	London	16-19	Male	Single	Step-son/daughter
2	1	1	Scotland	35-39	Male	Single	Household Representative
3	1	1	Yorks and the Humber	35-39	Female	Married/Civil partner-ship	Household Representative
3	1	2	Yorks and the Humber	35-39	Male	Married/Civil partner-ship	Spouse
3	1	3	Yorks and the Humber	5-10	Male	Single	Step-son/daughter

household	family	person	region	age_group	sex	marital_status	rel_to_hrp
4	1	1	Wales	0-4	Male	Single	Son/daughter (incl. adopted)
4	1	2	Wales	60-64	Male	Married/Civil partner-ship	Household Representative
4	1	3	Wales	55-59	Female	Married/Civil partner-ship	Spouse
4	2	3	Wales	30-34	Female	Single	Son/daughter (incl. adopted)

The first five people in the FRS all belong to the same household (household 1); they also all belong to the same family. This family comprises a married middle-aged couple plus their three children, one of whom is a stepson.

The second household (household 2) comprises only one person – a single middle-aged male. The third household comprises another married couple, this time with two children.

Superficially the fourth household looks similar to households 1 and 2: a married couple plus their daughter. The difference is that this particular married couple is nearing retirement age, and their daughter is middle-aged. Consequently, despite being a child of the married couple, the middle-aged daughter is treated as a separate ‘family’ (family 2 in the household). This is because the FRS (and Census) define a ‘family’ as a couple plus any ‘dependent’ children. A dependent child is defined as a child who is either aged 0-15 or aged 16-19, unmarried and in full-time education. All children aged 16-19 who are married or no longer in full-time education are regarded as ‘independent’ adults who form their own family unit, as are all children aged 20+.

The inclusion of all persons in a household allows us more flexibility in the types of research question we can answer. For example, we could explore how the likelihood of a woman being in paid employment `WorkStatus` is influenced by the age of the youngest child still living in her family (if any) `fam_youngest`.

In the FRS (and Census), a “family” is defined as a couple and any “dependent” children. Dependent children are defined as those aged 0–15, or aged 16–19 if unmarried and in full-time education.

2.2.3 Explore the Distribution of Your Outcome Variable

Before starting your analysis, it is critical to know the type of scale used to measure your outcome variable: is it categorical or continuous? Here we will start off by exploring a continuous variable which can then turn into a categorical variable (e.g. top earners: yes or no).

We explore the income distribution in the UK by first looking at the low and high end of the distribution ie. What sorts of people have high (or low) incomes?

In the FRS each person's annual income is recorded, both gross (pre-tax) and net (post-tax). This income includes all income sources, including earnings, profits, investment returns, state benefits, occupational pensions etc. As it is possible to make a loss on some of these activities, it is also possible (although unusual) for someone's gross or net annual income in a given year to be negative (representing an overall loss).

Task: Load the FRS dataset into your R environment, if it's not already loaded, and inspect the data.

Open the dataset in RStudio's **Data Viewer** to explore its structure, including the `income_gross` and `income_net` variables.

```
# Open the data in the RStudio Viewer
View(frs_data)
```

in the **Data Viewer** tab, scroll horizontally to locate the `income_gross` and `income_net` columns. If columns are listed alphabetically, they will appear near other attributes that start with "income."

You should notice two things:

- Incomes are recorded to the nearest £, NOT in income bands.
- Dependent children almost all have a recorded income of £0.

This second observation highlights the somewhat loose wording of our question above (*What sorts of people have high (or low) incomes?*). To avoid reaching the somewhat banal conclusion that those with the lowest of all incomes are almost all children, we should re-frame the question more precisely as *What sorts of people (excluding dependent children) have low incomes?*

Task: Determine the Scale of the Outcome Variable.

```
# Summarize income variables
summary(frs_data$income_gross)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
-354848	52	12740	17305	23712	1127360

```
# Summarize income variables
summary(frs_data$income_net)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
-358592	0	12012	14204	20384	1110928

Task: Exclude Dependent Children.

You need to select all cases (persons) that are independent, that is where the variable `dependent` has values different from `!= "Dependent"` or equal `== "Independent"`.

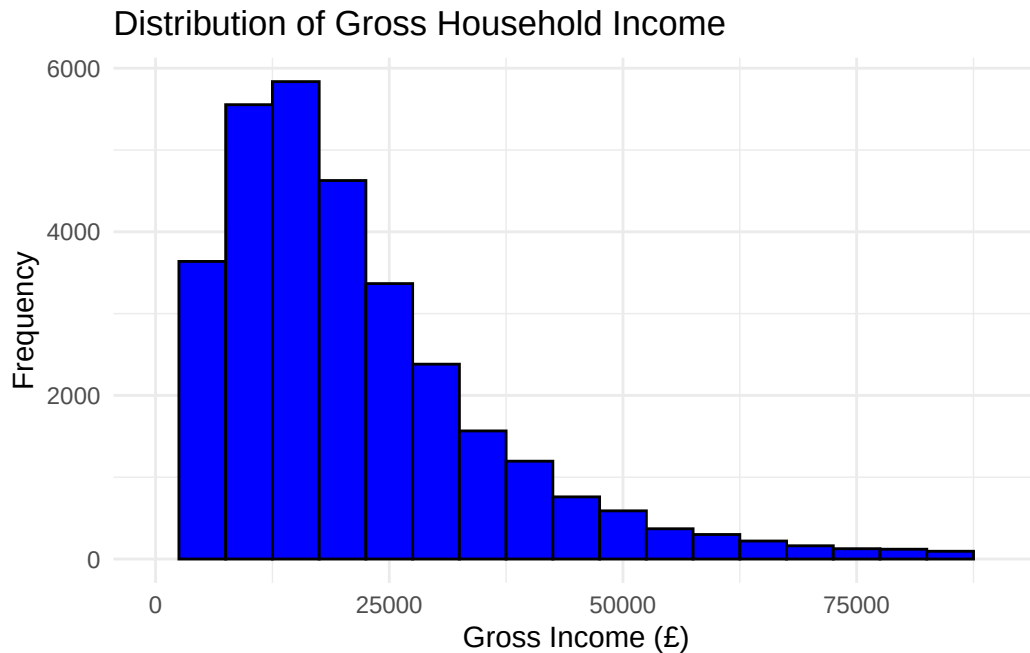
```
# Filter to include only independent persons
frs_independent <- frs_data %>% filter(dependent != "Dependent")
```

Task: Create a basic histogram (a visualisation lecture is scheduled later on).

The income variables in the FRS are all scale variables so a good starting point is to examine its distribution looking at a histogram of `income_gross`.

```
library(ggplot2)

ggplot(frs_independent, aes(x = income_gross)) +
  geom_histogram(binwidth = 5000, fill = "blue", color = "black") +
  labs(
    title = "Distribution of Gross Household Income",
    x = "Gross Income (£)",
    y = "Frequency"
  ) +
  xlim(0, 90000) +
  theme_minimal()
```



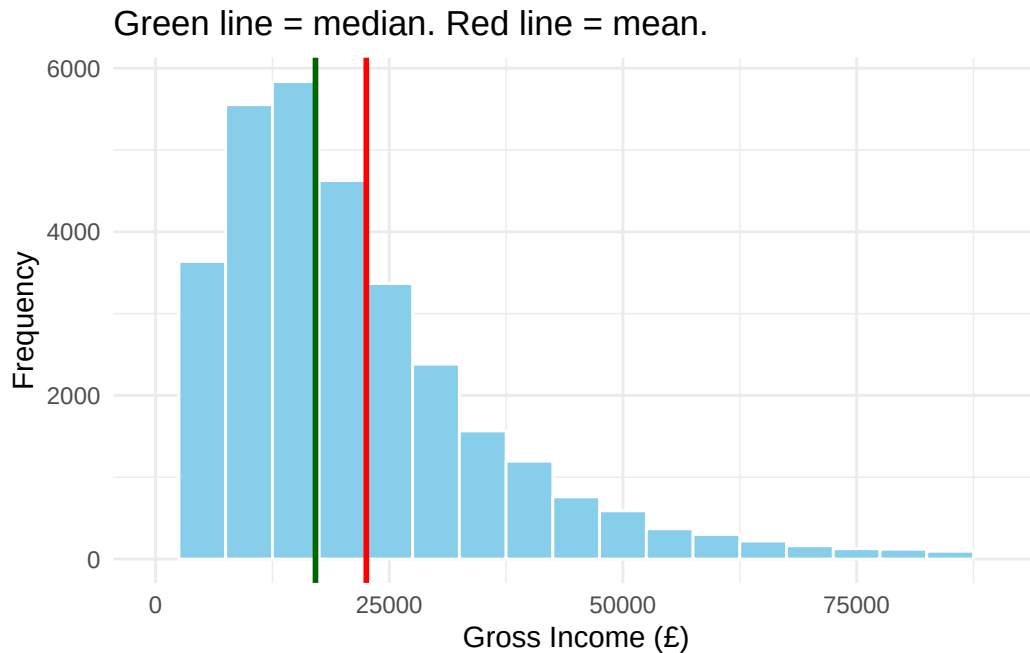
You should see the histogram below. It reveals that the income distribution is very skewed with few people earning high salaries and the majority earning just over or less 35,000 annually.

Why this matters: the mean and the median disagree.

Look back at the `summary()` output. The mean gross income is about £22,544, but the median is about £17,108 — a difference of more than £5,000. Both are “the average”. Both are correct. So which one should you report?

The histogram answers it. Let’s draw the two numbers onto it:

```
ggplot(frs_independent, aes(x = income_gross)) +
  geom_histogram(binwidth = 5000, fill = "skyblue", color = "white") +
  geom_vline(xintercept = median(frs_independent$income_gross), colour = "darkgreen", linewidth = 1) +
  geom_vline(xintercept = mean(frs_independent$income_gross), colour = "red", linewidth = 1) +
  labs(
    title = "Green line = median. Red line = mean.",
    x = "Gross Income (£)",
    y = "Frequency"
  ) +
  xlim(0, 90000) +
  theme_minimal()
```



`geom_vline()` simply draws a vertical line at the position you give it. Here we give it the median and the mean.

The green median line sits in the middle of the tall bars, where most people actually are. The red mean line sits noticeably to the right, in a region where comparatively few people are. Nobody moved it there deliberately: the mean is dragged upwards by the long tail of very high earners off to the right of the chart (remember, the highest income in this dataset is over £1,000,000). The median cannot be dragged like this, because it only cares about who is in the middle of the queue, not how rich the richest person is.

Task: Which of the two lines better describes “a typical person’s income” in this dataset? This is not a trick question — look at where the bars are.

This is why income, house prices and similar skewed variables are almost always reported as medians, and it is a point worth making in your assignment if you use one of them. It is also a warning: **whenever you report a mean, look at the histogram first**. If the distribution has a long tail, the mean is describing the tail as much as it is describing your typical case.

Task: Adopt a regrouping strategy.

You can also cross-tabulate gross (or net) income with any of the other variables in the FRS to your heart’s content – or can you?

Again, here is important to recall that the income variables in the FRS are all ‘scale’ variables; in other words, they are precise measures rather than broad categories. Consequently, every single person in the FRS potentially has their own unique income value. That could make for

a table c. 44,000 rows long (one row per person) if each person has their own unique value. The solution is to create a categorical version of the original income variable by assigning each person to one of a set of income categories (income bands). Having done this, cross-tabulation then becomes possible.

But which strategy to use? Equal intervals, percentiles or ‘ad hoc’. Here I would suggest that ‘ad hoc’ is best: all you want to do is to allocate each independent adult to one of three arbitrarily defined groups: ‘low’, ‘middle’ and ‘high’ income. **Define Low and High Income Thresholds**

Define thresholds for income categories:

- Low-income threshold: £_____
- High-income threshold: £_____

Task: Create a New Variable Based on Regrouping of Original Variable.

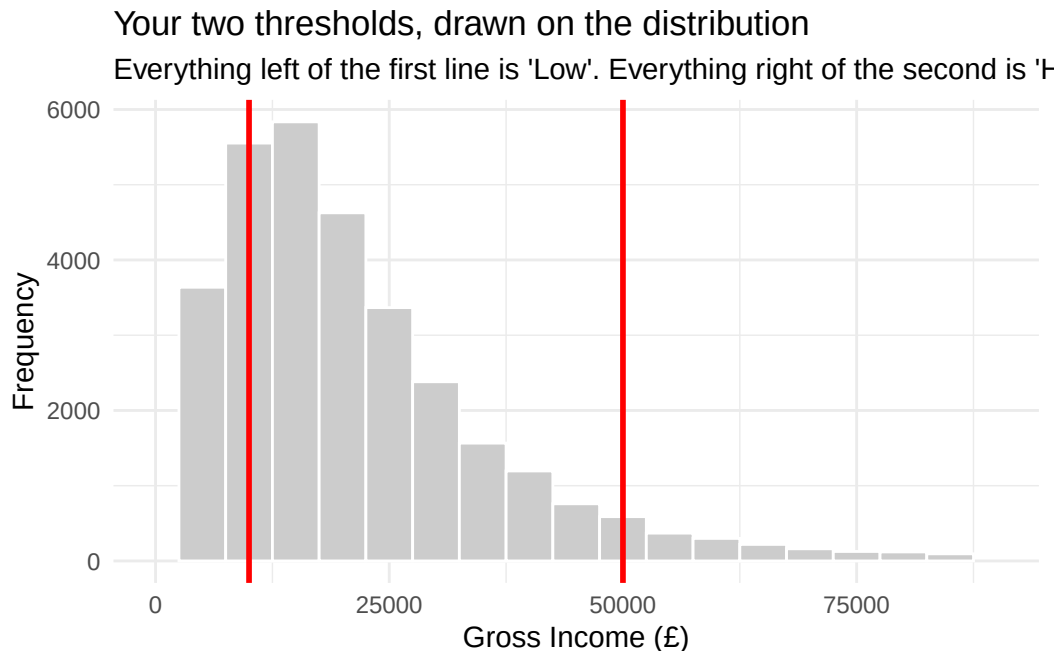
Recode `income_gross` into categories based on the chosen thresholds.

```
# Define thresholds for income categories
LOW_THRESHOLD <- 10000 # Replace with the upper limit for low income
HIGH_THRESHOLD <- 50000 # Replace with the lower limit for high income

# Define income categories based on thresholds
frs_independent <- frs_independent %>%
  mutate(income_category = case_when(
    income_gross <= LOW_THRESHOLD ~ "Low",
    income_gross >= HIGH_THRESHOLD ~ "High",
    TRUE ~ "Middle" ))
```

Before looking at the results, it is worth seeing what you have just done to the data. The two thresholds are simply two lines drawn across the distribution:

```
ggplot(frs_independent, aes(x = income_gross)) +
  geom_histogram(binwidth = 5000, fill = "grey80", color = "white") +
  geom_vline(xintercept = LOW_THRESHOLD, colour = "red", linewidth = 1) +
  geom_vline(xintercept = HIGH_THRESHOLD, colour = "red", linewidth = 1) +
  labs(
    title = "Your two thresholds, drawn on the distribution",
    subtitle = "Everything left of the first line is 'Low'. Everything right of the second is 'High'",
    x = "Gross Income (£)",
    y = "Frequency"
  ) +
  xlim(0, 90000) +
  theme_minimal()
```



Notice how much of the population falls between the lines, and how little falls beyond the right-hand one. Now change `LOW_THRESHOLD` to 15000, re-run both chunks, and watch the red line move and the “Low” group grow.

Task: There is no correct answer here, and that is the point. You chose those two numbers, and your choice decides who counts as poor and who counts as rich. If you recode a variable like this in your assignment, you must say what thresholds you used and why — the marking criteria ask for exactly that.

The `mutate()` function in R, from the **dplyr** package, is used to add or modify columns in a data frame. It allows you to create new variables or transform existing ones by applying calculations or conditional statements directly within the function.

Explanation of the code

- `frs_independent %>%`: The pipe operator `%>%` sends `frs_independent` into `mutate()`, allowing us to apply transformations without reassigning it repeatedly.
- `mutate()`: Starts the transformation process by defining new or modified columns.
- `income_category = case_when(...)`:
 - This creates a new column named `income_category`.
 - The `case_when()` function defines conditions for assigning values to this new column.
- `case_when()`:
 - `case_when()` is used here to assign categorical labels based on conditions.

- `income_gross <= LOW_THRESHOLD ~ "Low"`: If `income_gross` is less than or equal to `LOW_THRESHOLD`, `income_category` will be labeled “Low.”
- `income_gross >= HIGH_THRESHOLD ~ "High"`: If `income_gross` is greater than or equal to `HIGH_THRESHOLD`, `income_category` will be labeled “High.”
- `TRUE ~ "Middle"`: Any values not meeting the previous conditions are labeled “Middle.”

Task: Add some Metadata.

Define metadata for the new variable by labeling income categories.

```
# Add metadata by converting to a factor and defining labels

frs_independent$income_category <- factor(frs_independent$income_category,
                                           levels = c("Low", "Middle", "High"), labels = c("<= £10,000", "£10,001 - £49,999", ">= £50,000"))
```

Task: Check your work.

Examine the frequency distribution of the variable you have just created. Both variables should have the same number of missing cases, unless:

- Missing cases in the old variable have been intentionally converted into valid cases in the new variable.
- You forgot to allocate a new value to one of the old variable categories, in which case the new variable will have more missing cases than the old variable.

```
# Frequency distribution of income categories
table(frs_independent$income_category)
```

<= £10,000	£10,001 - £49,999	>= £50,000
8584	22981	2271

After preparing the data, use cross-tabulations to compare income levels across demographic groups.

```
# Cross-tabulate income category by age group, nationality, etc.
table(frs_independent$income_category, frs_independent$age_group)
```

	16-19	20-24	25-29	30-34	35-39	40-44	45-49	50-54	55-59	60-64
<= £10,000	373	680	492	558	474	511	554	652	781	826
£10,001 - £49,999	263	1241	1802	2056	2052	1948	1995	1967	1749	1772
>= £50,000	1	8	59	186	314	331	334	356	237	177

	65-69	70-74	75+
<= £10,000	773	744	1166
£10,001 - £49,999	2073	1554	2509
>= £50,000	144	56	68

Explore income distribution across different regions.

```
# Cross-tabulate income category by region
table(frs_independent$income_category, frs_independent$region)
```

	East Midlands	East of England	London	North East	North West
<= £10,000	562	665	740	357	878
£10,001 - £49,999	1550	1855	1850	979	2347
>= £50,000	135	245	367	48	174

	Northern Ireland	Scotland	South East	South West	Wales
<= £10,000	874	1212	895	588	399
£10,001 - £49,999	2305	3234	2563	1707	971
>= £50,000	123	322	367	149	63

	West Midlands	Yorks and the Humber
<= £10,000	744	670
£10,001 - £49,999	1892	1728
>= £50,000	164	114

Tips for Cross-Tabulation

- Place the income variable in the columns.
- Add multiple variables in the rows to create simultaneous cross-tabulations.

2.3 Practice: From Sample to Population

Everything we have done so far describes **the people in our dataset**. But nobody actually cares about the 33,836 adults in the FRS file. We care about adults in the UK. The FRS is a

sample: the Department for Work and Pensions knocked on a set of randomly chosen doors, and these are the people who answered.

The key question: our sample says the mean gross income is about £22,544. If the DWP had knocked on a *different* set of doors, would they have got the same number?

2.3.1 Sampling Variability

Let's find out. For the next few minutes, pretend the 33,836 people in the FRS **are** the whole population. That lets us do something the DWP cannot: draw a small sample out of it, look at the answer, then put everybody back and draw another one.

```
mean(sample(frs_independent$income_gross, size = 500))
```

```
[1] 21416.51
```

Read that from the inside out: `sample(...)` picks 500 incomes at random, and `mean(...)` averages them.

Task: Run that line three or four more times. Write down the answer each time.

You will get a different number every time — usually somewhere between about £21,000 and £25,000, but essentially never exactly £22,544. Nobody made a mistake. This is **sampling variability**: the unavoidable wobble that comes from a sample being only a part of the whole.

Now let R do it a thousand times and collect all the answers:

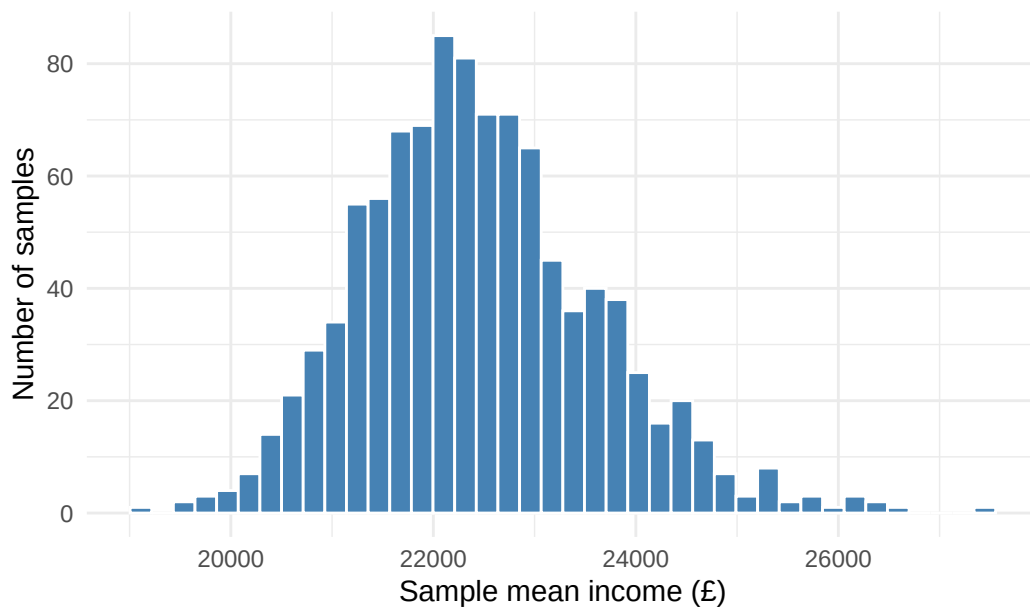
```
many_means <- replicate(1000, mean(sample(frs_independent$income_gross, size = 500)))
```

`replicate(1000, ...)` does what its name suggests: it runs whatever is in the brackets 1000 times over and keeps every result. `many_means` now holds 1000 numbers, each the average income of a different imaginary survey.

```
means_table <- data.frame(sample_mean = many_means)

ggplot(means_table, aes(x = sample_mean)) +
  geom_histogram(bins = 40, fill = "steelblue", color = "white") +
  labs(
    title = "1000 samples of 500 people: what mean income did each one report?",
    x = "Sample mean income (£)",
    y = "Number of samples"
  ) +
  theme_minimal()
```

1000 samples of 500 people: what mean income did each one have?



Two things are worth noticing. The histogram is centred on the true value of about £22,500 — samples are not systematically wrong, they are wrong in *both* directions roughly equally. And it is far narrower than the income distribution you plotted earlier: individual incomes ran from large losses to over a million, but *sample means* nearly all land within a few thousand pounds of each other.

2.3.2 The Standard Error

The width of that histogram is the thing we want, because it says how far a typical sample lands from the truth. It has a name: the **standard error**.

```
sd(many_means)
```

```
[1] 1160.505
```

About £1,200. In other words, a sample of 500 people typically misses the true mean income by around £1,200.

Be careful not to confuse this with the standard deviation of income, which was about £27,000. That one describes how much **people** differ from each other. The standard error describes how much **samples** differ from each other. Mixing up the two is one of the most common errors in student reports.

In real life you only ever have one sample, so you cannot build the histogram above. Happily you do not need to — the standard error can be worked out from your single sample, as the standard deviation divided by the square root of the sample size:

```
sd(frs_independent$income_gross) / sqrt(500)
```

```
[1] 1227.262
```

Compare that with the `sd(many_means)` value. They agree closely: the formula predicts the width of a histogram we never had to build.

2.3.3 What a P-value Means

From next week onwards, almost every result you produce will come with a **p-value** attached, so it is worth knowing now what one is.

Suppose we ask whether men and women have different incomes. In our data they clearly do:

```
frs_independent %>%  
  group_by(sex) %>%  
  summarise(mean_income = mean(income_gross, na.rm = TRUE), n = n())
```

```
# A tibble: 2 x 3  
  sex      mean_income      n  
  <chr>      <dbl> <int>  
1 Female      18134.  17858  
2 Male       27473.  15978
```

A gap of roughly £9,300. But we have just spent half an hour establishing that samples wobble, so before concluding anything about the UK we have to rule out the boring explanation: **could a gap this large have turned up purely through the luck of who got sampled, even if men and women earned the same across the country?**

That is the question a p-value answers, and it is the only question a p-value answers:

A p-value is: if there were genuinely no difference in the population, how often would sampling alone hand us a gap at least as large as the one we observed?

A small p-value means “hardly ever, so sampling luck is a poor explanation for what I am seeing”. A large p-value means “quite often, so I cannot rule out that this is just noise”. The statement being tested — that there is genuinely no difference in the population — is called the **null hypothesis**.

```
t.test(income_gross ~ sex, data = frs_independent)
```

Welch Two Sample t-test

```
data: income_gross by sex
t = -30.94, df = 26135, p-value < 2.2e-16
alternative hypothesis: true difference in means between group Female and group Male is not 0
95 percent confidence interval:
 -9930.465 -8747.247
sample estimates:
mean in group Female    mean in group Male
      18134.42           27473.28
```

The squiggle `~` is read as “by” or “depends on”, so `income_gross ~ sex` means “income, broken down by sex”. You will meet this notation constantly from next week, because it is how R writes down a model.

The p-value is reported as `< 2.2e-16`, R’s way of writing “smaller than 0.00000000000000022”. Sampling variation is essentially impossible as an explanation for a gap this large in a sample this size, so we conclude that a gender income gap exists in the population — not just in our spreadsheet. By convention, a p-value below 0.05 is called **statistically significant**.

Two warnings to carry into your assignment:

- A p-value is **not** the probability that your finding is correct. It is calculated *assuming* there is no real difference, so it cannot tell you how likely that assumption is.
- A small p-value does **not** mean the effect is large or important. With 33,836 people, even a trivial £50 gap would come out “highly significant”. Always report the size of an effect alongside its p-value.

Going further (optional, in your own time): there is more to say about all of this — what a confidence interval really means, why “not significant” does not mean “no difference”, and why Census data needs handling differently. It is written up in [Samples, Uncertainty and P-values](#). You do not need it today, but it will help when you write your report.